

Extermination Protocol - MSX-BASIC 10 Liner - Line-by-Line Analysis

This program is a classic MSX-BASIC arcade-style game using sprites, sound, keyboard control, and character redefinition.

Line 1 – Initialization

```
DEFINT A-Z  
KEY OFF  
COLOR 12,1,4  
SCREEN 1,1,0  
WIDTH 32
```

- **DEFINT A-Z**: All variables are integers (faster execution).
- **KEY OFF**: Disables function key display.
- **COLOR 12,1,4**: Dark green text, black background, blue border.
- **SCREEN 1,1,0**: Graphics 1 mode (32x24 tile-based display), 16x16 sprites, key click off.
- **WIDTH 32**: 32 text columns.

Character Redefinition

The loop reads hexadecimal data and writes it directly into VRAM using `VPOKE`. This redefines ASCII character 87 and copies it into the active pattern table.

Sprite Creation

```
C$ = CHR$(&h3C)  
S2$ = C$ + C$  
SPRITE$(7) = S2$
```

Creates a simple 8x8 sprite pattern.

Game Setup

- **D=2**: Initial direction.
- **ON SPRITE GOSUB 6**: Hardware sprite collision detection → jump to line 6 (Game Over).
- **GOSUB 8**: Initialize game state.
- **PLAY "T120V12"**: Set music tempo and volume.

Line 2 – Mine Collection Detection

```
IF INT((HX+8)/8)=X AND INT((HY+8)/8)=Y THEN ...
```

- **(HX, HY)**: Player sprite position (pixels).
- **(X, Y)**: Mine position (character grid).

The sprite coordinates are divided by 8 because screen characters are 8x8 pixels.

If both positions match:

- The mine is erased
- Sound beep plays
- Score increases
- Score display updates

Line 3 – Sprite Update and Movement

```
PUTSPRITE 0,(EX,EY),12,AN+D  
PUTSPRITE 2,(HX,HY),10,AN+4
```

- Sprite 0 → Enemy
- Sprite 2 → Player
- **AN** toggles animation frame
- **D** controls direction the enemy is facing

Keyboard Control

```
J = STICK(0)
```

In this context, **STICK(0)** reads keyboard directional input.

Movement logic example:

```
HY = HY + 2*(J=1 AND HY>8)
```

In MSX-BASIC:

- True = -1
- False = 0

This produces smooth 2-pixel movement.

Enemy AI

```
EX = EX - (EXHX)  
EY = EY - (EYHY)
```

The enemy always moves 1 pixel toward the player on both axes.

Timed Mine Spawn

```
IF TIME > 430 THEN GOSUB 9
```

After a certain time, a new mine appears.

Line 4 – Animation Control

```
TC = TC + 1  
IF TC < 4 THEN Z=0 ELSE TC=0:AN=1-AN
```

Toggles animation frame every 4 cycles.

```
IF EX < HX THEN D=2 ELSE D=0
```

Updates sprite direction based on enemy position.

Line 5 – Main Loop

```
GOTO 2
```

Returns to collision detection and continues the game loop.

Line 6 – Game Over

Triggered by hardware sprite collision.

Displays flashing “GAME OVER” text with sound effect.

If current score $\geq$ high score, updates High Score.

Line 7 – Restart

```
IF NOT STRIG(0) THEN 7
```

Waits for key/button press.

- Clears screen
 - Reinitializes game
 - Returns to main loop
-

Line 8 – Game Initialization

- Resets player and enemy positions
- Resets score
- Clears sprites

Random Tree Generation

```
FOR I=0 TO 8  
X1=INT(RND(1)*16)*2  
Y1=INT(RND(1)*11)*2  
LOCATE X1,Y1  
NEXT
```

Draws random trees on the screen.

Sprites are re-enabled and intro sound plays.

Line 9 – New Mine Generation

- Erases previous mine
- Generates new random position
- Plays PSG sound using SOUND commands
- Displays mine character

If flag B=1, leaves a sprite trail effect.

Line 10 – Character and Sprite Redefinition Data

This line contains the hexadecimal data used for redefining character patterns and sprite graphics. The values following the `DATA` statement are read earlier in the program and written directly into VRAM using `VPOKE`, defining custom visual elements used by the game.

Variable Definitions

- **EX / EY** = Enemy position
 - **HX / HY** = Hero position
 - **M** = Disabled mines
 - **N** = Sprite number
 - **S** = Score
 - **H** = High score
 - **TC and AN** = Control the animation (walking steps)
 - **D** = Controls the direction the enemy is facing
 - **C** = Background color
 - **T** = Stage
 - **B** = Boolean that activates (when TRUE) the red mine/sprite display
 - **B2** = Boolean that controls the counting of recovered mines (one at a time)
 - **B3** = Boolean that controls the mine display (when it is yellow)
 - **U / W** = Tree positions
 - **X / Y** = (Randomly generated) position of the next mine
 - **X2 / Y2** = Temporary values of variables X and Y
 - **V** = Speed / Level
-

Overall Game Behavior

1. Player moves using keyboard
2. Enemy constantly chases player
3. Mines appear periodically
4. Disabling mines increases your score
5. Enemy collision triggers Game Over
6. High Score is preserved
7. Gradual difficulty progression through increasing levels
8. Infinite stages with continuous gameplay
9. Uses sprites, hardware collision, PSG sound, animation, and character redefinition

